PARTE 1: USAR LAS COLECCIONES DE JAVA

EJERCICIO 1 (2 PUNTOS). Usar List<T> y alguna de las clases que la implementan. Supongamos que nos encargan realizar un software que sea capaz de detectar preguntas copiadas en trabajos. Dados dos textos (que son la contestación a una pregunta), tendremos que pensar como medir lo parecidos o diferentes que son. Lejos de ser algo novedoso, estas técnicas se utilizan habitualmente en motores de búsqueda (google y muchos más), y no solo para buscar páginas web, también en bases de datos, etc..

Normalmente se utiliza el término distancia o similitud. La distancia es una medida numérica que nos indica lo diferentes que son: cuanto mayor sea la distancia que guardan dos textos, más diferentes son. Esta medida puede normalizarse para que vaya de 0 a 1 (no siempre), donde 0 significa que son exactamente iguales y 1 significa que no tienen nada en común. La similitud es lo contrario, a mayor similitud más parecidos y a menor similitud más diferentes: similitud = 1.0 – distancia (si la distancia está normalizada).

Para experimentar, vamos a coger una pregunta de un trabajo de Economía, relacionada con los organismos de defensa del consumidor. La pregunta es esta: **¿Qué empresas están obligadas a adherirse a un sistema arbitral de consumo?** Y tenemos 3 respuestas:

> Texto1 = "Adherirse al sistema arbitral es voluntario e incluso pueden solicitar su adhesión dejando al margen ciertos conflictos. Aunque sin estar inscrita, podría aceptar este mecanismo en algún caso.

Otra respuesta es una copia a la que han cambiado el principio de la primera frase:

> Texto2 = "Unirse es voluntario e incluso pueden solicitar su adhesión dejando al margen ciertos conflictos. Aunque sin estar inscrita podría aceptar este mecanismo en algún caso."

Y otra respuesta es una nueva copia, pero ahora han cambiado el orden, primero escriben la segunda frase y luego la primera.

> Texto3 = "Aunque sin estar inscrita podría aceptar este mecanismo en algún caso pero adherirse al sistema arbitral es voluntario e incluso pueden solicitar su adhesión dejando al margen ciertos conflictos. "

Hay medidas (de distancia/similitud) que se fijan en el léxico. Si tomamos como elemento unidad la palabra, miden la coincidencia de estos elementos, o cuantas operaciones (insertar, eliminar, modificar) son necesarias para transformar un texto en el otro. Estos no detectarían copias que consistan en cambios de posición de trozos.
Otras distancias se centran en la semántica. Hay dos tipos: basadas en corpus y en conocimiento. El corpus consiste en un conjunto de textos de los que se extrae información estadística del lenguaje que se utiliza. Las semánticas basadas en conocimiento utilizan bases de datos de elementos del lenguaje y son los más efectivos y complejos de implementar.

Nosotros, usaremos este pretexto para practicar con arrays y diferentes colecciones de Java y vamos a calcular dos distancias: D1 es una distancia léxica denominada el algoritmo de

Levenshtein y otra D2 que es semántica basada en corpus, denominada el coeficiente de Jaccard. Pero para simplificar el ejercicio, no utilizaremos corpus.

Crea un programa que defina las 3 respuestas anteriores e implemente estos métodos:

a) REPASO: (0.75 PUNTOS) Calcular D1: Una versión de la distancia de Levenshtein (normalizada a 0-1). Utiliza solamente arrays y métodos de la clase **Arrays** .

```
DOUBLE coeficienteLevenshtein(String texto1, String Texto2)
    t1 ← Array de palabras de texto1 pasadas a minúsculas sin signos de puntuación
    t2 ← Array de palabras de texto2 pasadas a minúsculas sin signos de puntuación
    declara d[t1.longitud+1, t2.longitud+1]
    declara int i, j, coste
    para i desde 0 hasta t1.longitud haz d[i, 0] ← i; finpara
    para j desde 0 hasta t2.longitud haz d[0, j] ← j; finpara
    para i desde 1 hasta t1.longitud haz
        para j desde 1 hasta t2.longitud haz
            si t1[i-1] = t2[j-1] entonces coste ← 0 sino coste ← 1; finsi;
            d[i,j] ← minimo(
                        d[i-1, j] + 1,      // borrado
                        d[i, j-1] + 1,      // inserción
                        d[i-1, j-1] + coste   // sustitución
                    )
    finpara
finpara
devuelve minimo( 1,
                d[t1.longitud, t2.longitud]/maximo(1.0, ABS(t1.longitud - t2.longitud))
```

b) (0.75 PUNTOS) Calcular D2: El coeficiente de Jaccard es más sencilla de calcular si usas colecciones de Java. Por tanto ahora no utilices arrays (más que de forma auxiliar), mejor utiliza algunas listas por ejemplo, e investiga sus métodos para calcular lo que necesites.

double coeficienteJaccard(String texto1, String texto2) { ...

PASO1: Calcular:

a = Nº de palabras comunes en los dos textos.

b = Nº de palabras que están en texto1 y no en texto2.

c = Nº de palabras que están en texto2 y no en texto1.

PASO3: calcular y devolver $\dfrac{a}{a + b + c}$

c) (0.5 PUNTOS) El programa debe imprimir:

```
Texto1 y Texto2:
       Similitud Lexica (Levenshtein):  1 - D1
       Similitud Semantica (Jaccard):   1 – D2

Texto1 y Texto3: (lo mismo) …

Texto2 y Texto3: (lo mismo) ...
```

EJERCICIO 2 (2 PUNTOS). ITERADORES Y COMPARADORES.

a) (0.5 PUNTOS) REPASO: Define una clase llamada **Municipio** que contenga el nombre de una población y sus habitantes. (Crea constructores, getters, setters y sobreescribe toString().

b) (0.5 PUNTOS) Queremos manipular estos datos mediante una colección Lista. Crea una lista y añade estos y calcula la media de la población usando un iterador.

> **{"Ademuz", 1149} {"Ador", 1374} {"Adzaneta de Albaida", 1307}, {"Agullent", 2402} {"Alaquàs", 29838} {"Albaida", 5987} {"Albal", 16029}** ...

c) (0.5 PUNTOS) Modifica la clase Municipio para que implemente la interfaz Comparable. Haz que un municipio sea menor que otro si su población es menor, mayor si lo es su población o igual solo si su población es igual y también lo es su nombre. Muestra los municipios de la colección ordenados (Si lo pides a la colección, usará compareTo() de la clase que almacene) recorriéndola con un bucle while y un iterator.

d) (0.5 PUNTOS) Ahora imagina que queremos los elementos ordenados por un criterio distinto al que tiene la clase, por ejemplo por orden alfabético. Crea un objeto de tipo Comparator y pide a la clase Collection que te ordene la lista por este nuevo criterio, y la imprimes mediante un bucle for-each.

e) OPCIONAL (+0.5). ¿Cómo definirías el comparador del apartado b) mediante una función lambda?

EJERCICIO 3 (1 PUNTO). Usar Pilas. Una pila (stack) es una lista LIFO, una lista normal en la que solamente utilizas dos operaciones push() para añadir un elemento y pop() para sacar un elemento. Como ejemplo de utilización de pilas vamos a hacer un programa que sea capaz de leer un string que contenga una serie de operaciones parentizadas y sea capaz de realizar el cálculo que indique la expresión.

2 * 3 + 4 → No se puede salvo que crees un compilador

((2 * 3) + 4) → podemos hacerlo sin compilador (cada operación un paréntesis)

E. W. Dijkstra desarrolló un algoritmo en 1960 que lo hace usando dos pilas, una pila de operandos (valores) y otra pila de operaciones. Una expresión contiene paréntesis, operandos (números) y operaciones. Recorriendo la expresión de izquierda a derecha y extrayendo cada uno de estos elementos uno a uno se hace lo siguiente:

- si encuentras un número: push(numero) en la pila de operandos.
- Si encuentras un operador: push(operador) en la pila de operaciones.
- Si encuentras un paréntesis izquierdo "(": lo ignoras.
- Si encuentras un paréntesis derecho: operacion= operaciones.pop(), sacar de la pila de operandos los valores que necesite la operación, realizar el cálculo y el resultado se vuelve a añadir a la pila de operandos.

Cuando se acaben los elementos de la expresión, si está escrita de forma correcta, en la pila de operandos quedará un valor que es el resultado de calcular la expresión.

> **Nota**: si no separas con espacios cada elemento de la expresión, necesitarás un analizador léxico que reconozca los elementos (llamados tokens) admitidos en el lenguaje (números, operaciones y paréntesis en este caso).
>
> **Nota**: si no encierras entre paréntesis cada expresión, necesitarías un traductor (intérprete o compilador) que coge los tokens (elementos) de la expresión, construye un árbol sintáctico de la expresión (lo controla la sintaxis del lenguaje) y lo transforma en operaciones que pueda implementar, por ejemplo en pilas de operaciones y valores.

Crea el programa en Java que lea la expresión de la entrada estándar (si la hay) o pida un String (si no se indica). Ejemplos:

```
C:\> java U8E3 ( 1 + ( ( 2 + 3 ) * ( 4 * 5 ) ) )
101.0

C:\> java U8E3
Teclee la expresión: ( ( 1 + raiz2( 5.0 ) ) / 2.0 )
1.618033988749895
```

PARTE 2: CONSTRUIR TUS PROPIAS COLECCIONES

EJERCICIO 4 (1 PUNTO). IMPLEMENTA LISTAS ENLAZADAS. Examina el código de ejemplo de la unidad sobre listas enlazadas (página 58) y:

a) (0.25 PUNTOS) Define la interfaz que describa el TDA Lista que se implementa y le añades dos operaciones:

```
// Comprueba si 0 <= pos <= nº elementos y Cambia el elemento actual al indicado
// en pos y devuelve true. En otro caso, no lo hace y devuelve false
boolean irA(int posicion);

// Si inc es positivo intenta mover el elemento actual inc posiciones a la derecha
// Si no hay tantos elementos, se quedará en el último
// Si inc es negativo, intenta mover el elemento actual inc posiciones a izquierda
// Si no hay tantos elementos, se quedará en el primero
// En cualquier caso devuelve la posición absoluta en que queda
int moverA(int inc);
```

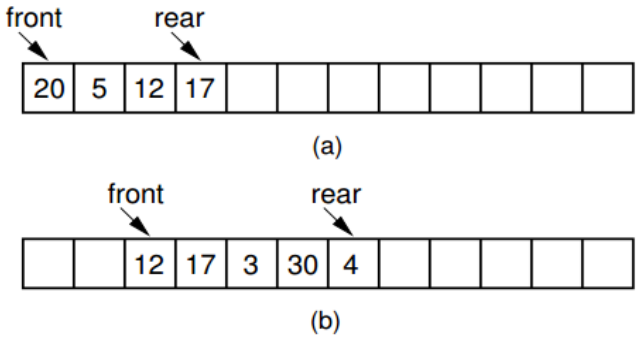
b) (0.75 PUNTOS) Además, modifica la clase y las implementas.

EJERCICIO 5 (1 PUNTO). IMPLEMENTA COLA DE ENTEROS CON ARRAYS. Las colas son una estructura secuencial como las listas (un elemento detrás de otro) pero que restringe las posiciones por donde se añaden o eliminan los elementos que la forman. En una cola los elementos se añaden siempre por el final de la cola (operación **encolar(elemento)**) mientras que siempre se sacan por la cabeza (operación **desencolar()**). Es una estructura FIFO "First-In, First-Out". Aquí está una posible interfaz para las Colas:

```
interfaz para Colas
        public void vaciar();
        public boolean encolar(elemento);
        public elemento desencolar();
        public elemento primero();
        public int longitud();
```

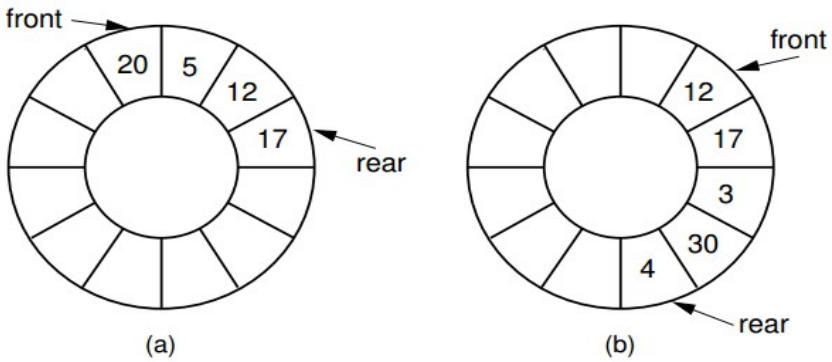
Vamos a usar un array normal de enteros para implementar una cola de números enteros. Si exigimos que los elementos ocupen las primeras posiciones, añadir y eliminar elementos nos obliga a desplazar los elementos que ya haya para no dejar huecos. Las operaciones las hacemos lentas (orden n, donde n es la longitud de la cola, es decir el número de elementos).

Para hacer las operaciones de **encolar()** y **desencolar()** de forma más eficiente, **encolar()** va a añadir por el final (como una lista), y desencolar() quitará elementos por el principio del array de forma que la cola se hace pequeña. Usando dos valores enteros que nos digan la posición de la cabeza y el fin de la cola podemos acelerar las operaciones. No dejamos huecos en el array y nos ahorramos mover elementos a cambio de actualizar donde está la cabeza y el fin de la cola después de cada cambio.



(a) La cola después de añadir los números 20, 5, 12 y 17
(b) La cola tras desencolar 20 y 5 y luego encolar 3, 30 y 4.

Otra mejora para que la cabeza y el fin de la cola puedan mover libremente por todas las posiciones del array consiste en hacerla circular actualizando los valores de la cabeza y el fin con la operación módulo.



(a) y (b) Las mismas operaciones de antes pero haciendo circular el array.

Nos queda por resolver un problema sutil. **¿Cómo saber si la cola está llena o vacía?** Podríamos pensar que **si fin < cabeza entonces está vacía**. Pero ¿Y si la cola está

completamente llena? Tendríamos la misma condición que si estuviese vacía, no podemos distinguir ambos casos. Así que la única solución es ir contando los elementos que tenemos en otra variable o crear un array con una posición más que el máximo de elementos a añadir y así fin < cabeza solo se produce si está vacía. El esqueleto de una implementación basada en Arrays:

(0.2 PUNTOS) Define la interfaz ColasInt

(0.6 PUNTOS) Define la clase ColaArrayInt que implemente la interfaz. Puedes usar este esqueleto.

```
class ColaArrayInt implements Colas {
        private static final int LONGITUD_POR_DEFECTO = 10;
        private int maxLongitud;        // Tamaño máximo
        private int cabeza;             // Índice del primero de la cola
        private int fin;                // Índice del último de la cola
        private int[] datos;            // Array que almacena elementos

        /** Constructores */
        public Cola() { this(LONGITUD_POR_DEFECTO); }
        public Cola(int max) {
                maxLongitud ←  max + 1;  // Un espacio extra
                fin ← 0;  cabeza ← 1;
                Inicializa el array datos
        }

        public void vaciar() { // poner el final a 0 y la cabeza a 1 }

        /** Añadir a la cola e */
        public void encolar(elemento) {
            precondición: (fin + 2) MODULO maxLongitud  DISTINTO cabeza
                           Si no se cumple, la cola está llena.
            fin = (fin + 1) % maxLongitud;     // Incremento circular
            datos[fin] = e;
        }

        /** Eliminar y devolver el primer elemento (cabeza) */
        public elemento desencolar() {
            precondicion: longitud() DISTINTO 0
                          Sino: "La cola está vacía";
            elemento ← datos[cabeza];
            cabeza ← (cabeza + 1) MODULO  maxLongitud;    // Incremento Circular
            devolver elemento;
        }

        /** @return primer valor */
        public elemento primero() {
            precondición: longitud() DISTINTO 0
                          sino se cunple: "La cola está vacía";
            devolver datos[cabeza];
        }

        /** @return Cantidad de elementos en la cola */
        public int longitud(){
            devolver ((fin + maxLongitud) - cabeza + 1) % maxLongitud;
        }
}
```

(0.1 PUNTOS) Haz un programa que defina una cola de enteros, la llene con 4 números

1,2,3,4. La imprima, saque 1 número, inserte otro y la vuelva a imprimir.

(0.1 PUNTOS) ¿Qué inconviente tiene esta forma de implementar TDA's? (piensa en que ocurriría si más adelante necesitamos una cola de doubles, luego una de Personas, luego una de Solicitudes, luego una de Alumnos, luego una de Facturas a àgar, de ….)

EJERCICIO 6 (1 PUNTO). IMPLEMENTA COLA CON ARRAYS QUE PERMITA ALMACENAR CUALQUIER OBJETO. Modifica el código anterior y:

a) (0.2 puntos) Define una interfaz Colas.
b) (0.6 puntos) Implementa una clase ColaArray que implemente la interfaz del apartado a) y sea capaz de almacenar cualquier objeto sin utilizar genéricos.
c) (0.1 puntos) Crea una cola de enteros, una cola de Strings y una cola de Personas, añade dos o tres elementos, las imprimes, desencola uno y las vuelves a imprimir.

Persona = {nombre, edad}

Nota: Cuando declares en la clase e inicialices en el constructor el array de datos, recuerda que debe poder almacenarse cualquier cosa, pora tanto para el código de la cola serán objetos lo más genéricos posibles.

d) (0.1 puntos) ¿Qué dos inconvenientes sigue teniendo nuestro código cuando utilicemos esta clase?

EJERCICIO 7 (1 PUNTO). IMPLEMENTA COLA CON ARRAYS QUE PERMITA ALMACENAR CUALQUIER OBJETO CON GENÉRICOS. Modifica el código anterior, define una interfaz Colas genérica y una clase ColaArray genérica que sea capaz de almacenar cualquier objeto. Crea una cola de enteros, una cola de Strings y una cola de Personas, y una cola sin indicar el genérico donde guardas personas, las imprimes, desencola uno y las vuelves a imprimir.

Persona = {nombre, edad}

Nota: Cuando declares en la clase e inicialices en el constructor el array de datos, utiliza esta expresión: datos `= (E[]) new Object[maxLongitud]; // Crea el array`

EJERCICIO 8 (OPCIONAL 2 PUNTOS). IMPLEMENTA COLA CON PRIORIDAD GENÉRICA USANDO ARRAYS. Modifica el código anterior del ejercicio 7 para que la cola con prioridad sea capaz de comparar entre sí los elementos que almacene. Por tanto los tipos genéricos añadidos deben implementar la interfaz **Comparable**. Esto asegura que el tipo tenga implementado el método **compareTo()** y el código de la cola podrá usar este método sin saber el tipo que va a almacenar. Hay que definir los elementos genéricos no como <T> sino como **<T extends Comparable<? super T>>** tanto en la interfaz como en la clase.

Lo único que cambia del código es la operación **encolar(T e) {...}**, porque ahora el elemento no tiene por qué quedarse en **datos[fin]**, puede ascender hacia la cabeza según implemente el método compareTo().

a) (0.2 PUNTOS) Define una interfaz **ColasConPrioridad** genérica con elementos comparables.

b) (0.8 PUNTOS) Implementa una clase **ColaConPrioridadA** que la implemente con arrays (por eso lo de la A al final) y que la operación **encolar()** la realice moviendo de forma secuencial el elemento añadido hasta su posición final comparándolo con los elementos desde el fin hasta la cabeza de la cola (esta operación tendrá orden n, donde n es la longitud de la cola).

c) (1 PUNTOS) Si eres capaz de hacer la operación **encolar()** con orden log2(n). **Pista:** es parecido a hacer una búsqueda secuencial o dicotómica, pero las posiciones van desde fin a cabeza y con operaciones modulo maxLongitud.

EJERCICIO 9 (1 PUNTO). IMPLEMENTA TABLA HASH GENÉRICA. A partir del apartado 8.5.4 de la unidad, define una interfaz genérica de una tabla hash y una clase también genérica que la implemente.

EJERCICIO 10 (OPCIONAL: +2 PUNTOS). IMPLEMENTA ARBOL BINARIO. Mira el apartado de árboles binarios y:

a) (0.5 Puntos) Define una interfaz y una clase genéricas de árboles binarios.

b) (0.5 Puntos). Define un iterador que recorra el árbol en inorden.

c) (0.5 Puntos). Define un iterador que recorra el árbol en preorden.

d) (0.5 Puntos) Define un iterador que recorra el árbol en postorden.

Todos los iteradores deben tener las operaciones: hayAnterior(), haySiguiente(), anterior(), siguiente().

EJERCICIO 11 (OPCIONAL: +4 PUNTOS). CÓDIGOS DE HUFFMAN

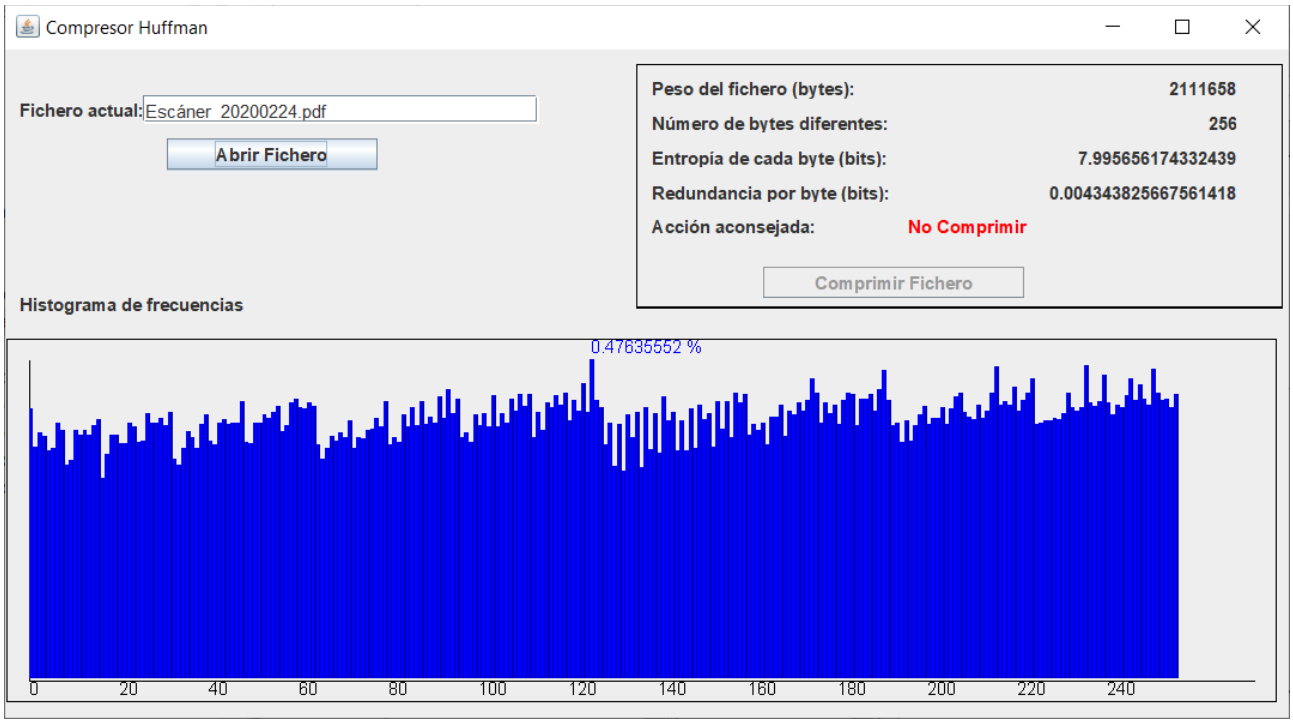
Cualquier fichero (sea de texto de binario) a fin de cuentas no es más que una serie de bytes formados por bits (0 y 1). A veces es posible modificar la forma en que se codifican estos bytes para conseguir que esa información pese menos (reducir la cantidad de bytes almacenados) este proceso se conoce como comprimir la información y es lo que hacen formatos como zip, .arj y otros.

Si pensamos en un fichero de texto con letras codificadas en ASCII, cada letra tiene 8 bits de peso asignados. Cada letra pesa un byte y cada byte tiene valores de 0 a 255. Una manera de comprimir consiste en cambiar estos códigos de tamaño fijo por otros códigos (bits) de tamaño variable, de forma que tengan menos bits, los valores (bytes) que más aparezcan en el fichero. Una forma sería usar un prefijo de tamaño diferente:

Byte que más aparece	Bits usados para escribirlo
B1	1
B2	01
B3	001
B4	0001

Pero una mejora, para diferentes bytes tener prefijos de igual longitud, se consigue utilizando la codificación de Huffman es una forma de aplicar esto.

PASO 1 (1 PUNTO): Debes hacer un programa GUI en Java que presente una interfaz en WindowBuilder similar a esta:



Cuando elijas un fichero, debe abrirlo como binario, recorrerlo y crear un histograma de los bytes que contiene, que no es más que un array de enteros de 256 valores donde en cada posición, cuentas las veces que aparece ese byte en el fichero (la posición 0 contiene cuantas veces aparece el byte 0, la posición 1 cuenta cuantas veces aparece el byte 1 en el fichero, …) así hasta la posición 255.

Luego hará un gráfico de barras que visualice el histograma. Para que se aprecie la distribución de bytes, se calcula la máxima frecuencia de aparición (el mayor valor del array de la tabla de frecuencias) y se grafican las barras de cada frecuencia de forma que la mayor coincida con la altura de la zona del gráfico menos 20 píxels. Al lado de la barra de la mayor

frecuencia, escribimos la frecuencia (en la figura 0.47...%).

El programa calcula la entropía de la información que hay en el fichero, que mide de media, cuantos bits en cada byte tienen realmente información, y cuantos bits de media son redundantes (sobran) y por tanto podrían ser prescindibles. La entropía máxima del fichero podría llegar a 8. Cuanto más cerca de 8 esté, menor será la redundancia (8 – entropía) y menos beneficioso es comprimirlo porque no conseguiremos bajar su peso de forma significativa (ya está comprimido). El programa debe desaconsejar comprimirlo mostrando un mensaje en rojo.

> **(0.5 PUNTOS) ACTIVIDAD:** Coge un fichero de texto sin comprimir (pero que no sean 4 letras, por ejemplo, copia el texto de esta práctica y lo pasas al bloc de notas y lo guardas como texto1.txt). Luego lo comprimes en texto2.zip. Ahora haz un programa que genere valores aleatorios entre 0 y 255 y los guardas de forma binaria en el fichero texto3.bin. Abre los tres ficheros con tu programa y muestra capturas de pantalla y es aconsejable o no comprimirlos. ¿Cuál tiene menor redundancia? ¿Un fichero aleatorio es similar a uno comprimido?

PASO 2 (1 PUNTO): DEFINIR UN ARBOL HUFFMAN.

Es un enlace a un nodo interno o Hoja que en realidad define un árbol binario donde los nodos hoja contienen un elemento E y un peso y los nodos intermedios solo contienen un peso que es la suma de sus hijos.

Construye una clase genérica para un nodo del árbol de Huffman que se llame **NodoBase<E>** que obligue a implementar las operaciones:

```
public boolean esHoja();   // Devuelve true si es un nodo hoja o false si no lo es
public int getPeso();      // Devuelve el peso del nodo (la frecuencia)
```

Ahora haz dos nuevas clases que deriven de la abstracta/interfaz llamadas NodoHoja y NodoInterno para diferenciar las hojas de los nodos intermedios que tienen diferente contenido.

NodoHoja	NodoInterno

Private E e; // elemento
private int peso; // peso
constructor(e, p)
getElemento() // devuelve el elemento e
getPeso() // devuelve el peso
esHoja() // devuelve true
	private int peso; // peso
	private NodoBase<E> hI; // hijo izq
	private NodoBase<E> hD; // hijo dere.
	constructor(NodoIzq, NodoDer, peso)
	getPeso() // devuelve el peso
	esHoja() // devuelve false
	getIzq() y getDer() // devuelve hijos

Un árbol Huffman será una referencia al nodo que hace de raíz de un árbol y sus elementos deben ser comparables. Lógicamente lo haremos genérico. Tendrá estos elementos:

```
class ArbolHuffMan<E> implements Comparable<ArbolHuffman<E>> {
        private NodoBase<E> raiz;
```

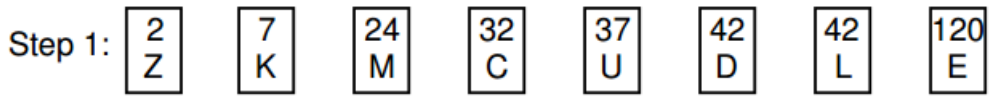
```
        // Constructores
        ArbolHuffman(E elemento, int peso) {
           raiz = nuevo nodoHoja<E>(elemento, peso);
        }

        ArbolHuffman( NodoBase<E> i, NodoBase<E> d, int peso) {
           raiz = nuevo NodoInterno<E>(i, d, peso);
        }

        public NodoBase<E> getRaiz() { ....

        public int getPeso() {  // devuelve el peso de la raíz....

        public int compareTo( ArbolHuffman<E> otro) {
                // devuelve: -1 si this.raiz.peso < otro.raiz.peso
                //           +1 si     "            >     "
                //            0 si son iguales
        }
}
```

PASO 3 (1 PUNTO): PROCESO DE CREAR ÁRBOLES DE HUFFMAN.

Imagina que tenemos la siguiente tabla de frecuencias donde solo aparecen los bytes con frecuencias mayores de cero de un fichero (los bytes que contiene un fichero no tienen porqué ser todos) y ten en cuenta que en la figura se ven letras en vez de el valor numérico de cada byte. La tabla de la figura indica que el carácter 'C' (o su valor numérico equivalente) aparece 32 veces, la 'D' aparece 42 veces, etc.

Letter	C	D	E	K	L	M	U	Z
Frequency	32	42	120	7	42	24	37	2

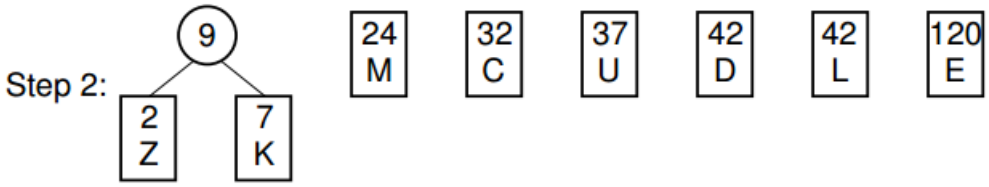
Crearemos un Árbol Huffman para cada entrada en la tabla que tenga un peso donde se almacena la frecuencia de los bytes con frecuencias mayores de cero. El nodo tiene el valor del byte y el peso (cantidad de veces que aparece en el fichero) y lo **añadimos a una cola con prioridad que almacena árboles binarios (nodos con: byte + freq + hi + hd + padre) ordenada por la frecuencia**. De esta forma, la cabeza de la cola (por donde desencolamos elementos) queda en los valores bajos: ← 2, 7, 24...



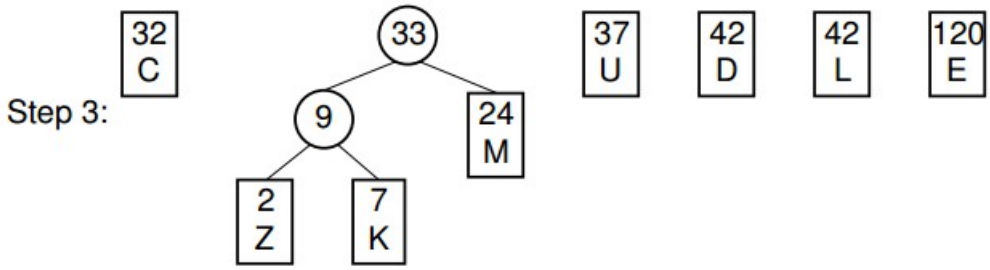

```
Mientras haya más de un elemento en la cola hacemos:
        desencolar en hijo izquierdo (variable iz)
        desencolar en hijo derecho (variable de)
        nuevo ← nuevo árbol binario(frec← iz.frec + de.frec, hijoI←  iz, hijoD←de);
        Encolar nuevo
```

```
FinMientras
```

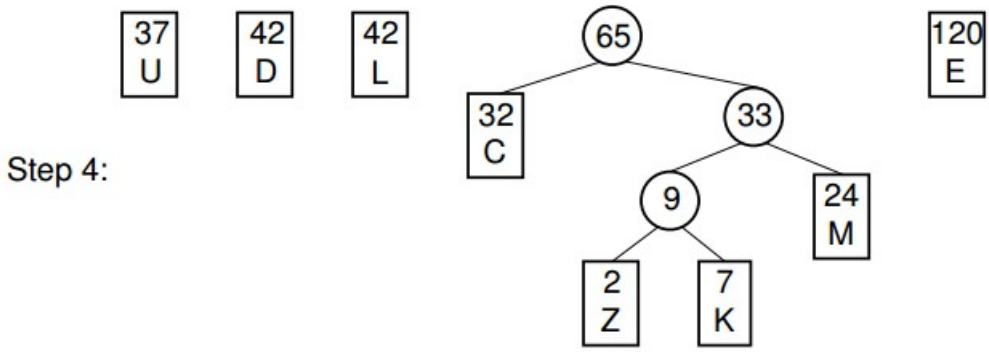
Este trozo de código irá construyendo un árbol binario (de Huffman). Por ejemplo, la primera vez combina el byte 'Z' y el byte 'K' en un árbol con peso 9 (7 + 2) que al final de la iteración se vuelve a encolar y debe quedar en la cabeza de la cola:



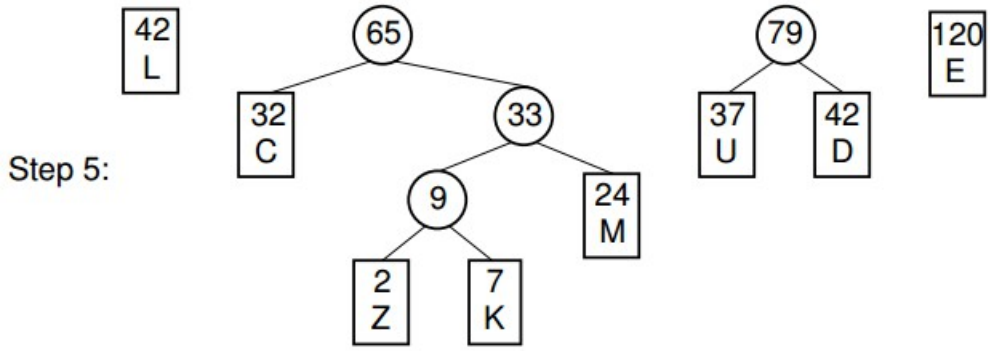
Como hay más de 1 elemento, vuelve a repetirse y en la segunda iteración la cola queda así:



En la tercera iteración del mientras:

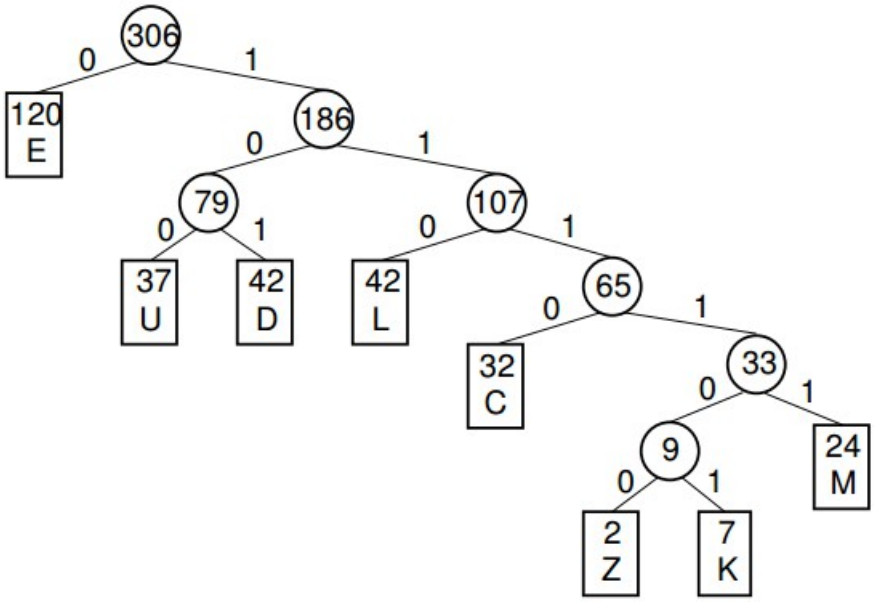


En la cuarta iteración:



Y al final, conseguimos tener la cola con prioridad con un solo elemento, con el árbol de

Huffman completado. En este caso se nos queda así:



Este proceso de construcción es un ejemplo de algoritmo de **greedy**. En cada paso, el algoritmo toma la decisión de mezclar los dos subárboles de menor peso, lo que da el resultado más óptimo. Si para cada nodo hoja, sacamos el recorrido hasta llegar desde la raíz a él, de forma que si vamos a la izquierda ponemos un 0 y si vamos a la derecha un 1, podemos generar los prefijos binarios por los que vamos a sustituir cada byte o letra o palabra:

Letter	Freq	Code	Bits
C	32	1110	4
D	42	101	3
E	120	0	1
K	7	111101	6
L	42	110	3
M	24	11111	5
U	37	100	3
Z	2	111100	6

PASO 4 (1 PUNTO). GENERAR NUEVO FICHERO CON INFORMACIÓN COMPRIMIDA.

Una vez que tenemos el árbol de Huffman construido, es fácil asignar los códigos a los bytes individuales. Nos hacemos un array de Strings de esta forma:

- **El valor de cada byte:** si el array es de 256 elementos (los String que almacenan estarán a null por defecto) podemos asociar el índice del array a cada byte posible (el byte 23 estará en la posición 23 del array, el 52 en la posición 52, etc.) y así accedemos rápidamente a su código y como los que están a null no van a utilizarse nunca, no hay problema.
- **Código binario de cada byte:** Un String con la cadena binaria (caracteres 0 y 1) que les corresponde en la nueva codificación Huffman.

Para calcular los códigos, comenzando en la raíz del árbol, asignamos '0' o 1 a cada enlace del árbol ('0' si nos vamos a la izquierda y '1' si nos vamos a la derecha). El código del byte que hay en cada nodo hoja será la ruta desde la raíz hasta el nodo hoja. El código para el byte 'E' será '0' porque la ruta hasta su nodo desde la raíz implica recorrer un enlace a la izquierda. El código para el byte 'K' es "111101" por el mismo motivo. Por tanto para obtener la tabla de códigos tendrás que recorrer el árbol y cuando estés en un nodo hoja, anotar el recorrido que llevas en la tabla de códigos del byte.

Para generar el fichero comprimido, y poder luego descomprimirlo y recuperar la información original debemos guardar no solo el mensaje comprimido sino la información que nos permita deshacer el proceso. Le vamos a dar el siguiente formato:

HUF	EXT	N	B	Tabla de frecuencias	Mensaje comprimido

Donde:

- **HUF (siempre 3 bytes)**: son los caracteres 'H' 'U' y 'F' una especie de marca que servirá a nuestro programa para reconocer que es un fichero generado por él. Sus ficheros acabarán con la extensión .huf pero podrían cambiarle la extensión a otro fichero e intentar reventarlo, para evitarlo, ponemos esta marca como una medida de protección.
- **EXT (siempre 4 bytes)**: Guardaremos 4 caracteres con la extensión que tenga el fichero original para que al descomprimir sepa que extensión ponerle (un "txt ", "png ", "pdf ", "docx", "jpeg", "html", …). Si la extensión original tiene solo 3 letras, añadimos un espacio en blanco al final para rellenar hasta 4.
- **N (siempre 8 bytes):** un long que contiene cuantos bytes pesa el mensaje original sin comprimir, para verificar que el fichero no está corrupto cuando al descomprimirlo recupere los bytes originales. Sigue descomprimiendo bytes hasta que consiga recuperar los que tenía el original. Si aún le quedan más de un byte (el relleno podría tener algunos bits más pero no más de 7 bits) los que obtiene no coinciden con los que tenía, y algo ha ido mal.
- **B (siempre 2 bytes)**: un short que contiene cuantos elementos hay en la tabla de frecuencias (máximo 256). Es decir, cuantos diferentes bytes tenía el fichero original. Desde 0 (si el fichero a comprimir estaba vacío) hasta 256 (todos los bytes posibles).
- **Tabla de frecuencias (B * 5 bytes, de 0 hasta como máximo 1280 bytes)**: la tabla de frecuencias nos va a permitir volver a reconstruir el árbol de huffman del fichero original. Cada entrada de la tabla tiene un byte (pesa 1 byte) y la frecuencia (la cantidad de veces que aparece en el fichero original) que es un int (4 bytes). Luego cada entrada pesa 5 bytes.
- **Mensaje comprimido**: los bytes con los datos codificados del mensaje original.

Como ves, cada fichero tiene que soportar cierto peso extra para poder deshacer el proceso de compresión. Como máximo el peso extra es 1280 + 8 + 2 + 4 + 3 = 1297 bytes como máximo. Para que el proceso de compresión sea efectivo, el fichero debe tener cierto peso, de lo contrario el ahorro que conseguimos lo perdemos por esta información.

Para generar el mensaje comprimido, debes volver a recorrer el fichero a comprimir desde el principio y sustituir cada byte por sus bits comprimidos. Yo te aconsejo que uses un String para ir añadiendo bits y sigas el siguiente proceso:

```
String s ← cadena vacía
int longitud, aDisco;
Mientras (haya bytes en el fichero)
        byte b ← siguiente byte del fichero
        s ← s + tablaCodigos[b]         // Añades al String, los bits del byte b
        longitud ←  codigos.length()
        while ( longitud >= 8 ) {
                aDisco ← convertir a byte ( primeras 8 letras de s )
                s <- s.subcadena(7)    // El resto de s pendiente de convertir
                longitud-= 8;
                guardar en el fichero .huf el byte
        }
}
si longitud > 0 {  // Si quedan bits por guardar
        mientras longitud modulo 8 != 0  {
                s ← s + "0";
                longitud ++;
        }  // Rellenar el último byte
        aDisco ← convertir a byte ( s )
}
```

PASO 5 (1 PUNTO). HACER EL PROCESO DE DESCOMPRIMIR. Cuando el programa comienza, el botón de descomprimir está desactivado. Cuando abras un fichero, si la extensión es .huf, el programa activa el botón de descomprimir. Si se pulsa el botón, el código debe:

SUBPASO 1. Validar que los 3 primeros bytes son 'H', 'U', 'F', de no ser estos, lanza un mensaje de advertencia en un diálogo, desactiva el botón de descomprimir y sale del proceso.
SUBPASO 2. Leer extensión, N (total de bytes a descomprimir) y B (entradas en la tabla de frecuencias).
SUBPASO 3. Crear un array de 256 elementos enteros y reconstruir la tabla de frecuencias usada al comprimir.
SUBPASO 4. Con la tabla de frecuencias generar el árbol de huffman usado para comprimir.
SUBPASO 5. Comienza a leer los bytes del fichero y que los bits te guíen por el inóspito árbol hasta encontrar nodos hoja, momento en el cual ya sabes el byte que debes

```
Long bytesDescomprimidos ← 0
nodoActual ← raíz del árbol huffman
MIENTRAS queden bytes por leer del fichero .huf AND bytesDecomprimidos < N HAZ
        b ← leer siguiente byte
        PARA i ← 1 HASTA 8 HAZ
                SI nodoActual no es una hoja
                        SI b AND 0x80 == 0  // Comprobar el bit
                        ENTONCES
                                nodoActual ← nodoActual.hijoIzquierdo()
                        SINO
```

```
                    nodoActual ← nodoActual.hijoDerecho();
                FINSI
            SINO  // es una hoja
                byteOriginal ← byte del nodo hoja nodoActual
                Escribir byteOriginal en el fichero descomprimido.
                BytesDescomprimidos ← bytesDescomprimidos + 1
                nodoActual ← raíz del árbol huffman
            FINSI
            b ← b DESPLAZADO A LA IZQUIERDA 1  // o bien multiplicar por 2
        FINPARA
    FINMIENTRAS
```

EJERCICIO OPCIONAL (+ 5 PUNTOS). CALCULADORA CIENTÍFICA QUE DIBUJE FUNCIONES f(x) = expresión con variables x.

La calculadora debe ser capaz de leer una expresión algebraica (que use variables x además de valores enteros y doubles, operaciones (+) (-) (*) (/) (^) y funciones (sen, cos, tan, asen, acos, atan, exp, log, ln), por ejemplo: 3, 4*x, 1/(x-1) , 3 * x^(x+1/x) - 4 , x/(1-sen(x)^2)

Para ser capaces de extraer las operaciones que necesitamos realizar a partir de un texto que contiene una expresión, necesitamos un compilador!! Un traductor que traslade lo que representa la expresión a unas operaciones realizables por el programa.

- **HARDWARE VIRTUAL**: ¿Qué usaremos para hacer los cálculos?

  RAM VIRTUAL DE DATOS

  Los datos los dejaremos en un array dinámico de doubles llamado datos, reservando la primera posición para la variable x. El resto de posiciones serán los valores numéricos que aparezcan en la expresión. Nuestra RAM donde está los datos será este array. Por ejemplo: 3+2/x genera estas entradas en el array datos: [x][3][2]

  RAM VIRTUAL DE SENTENCIAS

  Para realizar los cálculos, nuestro programa estará almacenado en una pila de sentencias llamada programa, usaremos una pila de operaciones y en vez de una CPU con registros, de forma que las operaciones se realizarán en orden infijo (la operación que se quiera ejecutar usará los datos de la pila), es como escribir "3 + 4" de la forma "+ 3 4" pero la operación estará en programa (la pila de sentencias) y los datos serán los del tope de la pila. Habrá sentencias que se aplican a dos datos y otras que se aplican a solo uno.

  ALU VIRTUAL

  Otra pila llamada alu será la encargada de ejecutar las operaciones. Las operaciones son:

OPERACION	DATO	DESCRIPCION
PUSH	**pos**	**alu.push(datos[pos])**
SUMA	**-----**	**alu.push(alu.pop() + alu.pop())**

```
CAMBIASIGNO    -----    alu.push( - alu.pop() )
RESTA          -----    alu.push( alu.pop()  - alu.pop() )
MULTIPLICA     -----    alu.push( alu.pop()  * alu.pop() )
DIVIDE         -----    num ← alu.pop()     den ← alu.pop()
                        SI den != 0
                        ENTONCES  alu.push( alu.pop()  / alu.pop() )
                        SINO
                           Warning →
                           SI num > 0
                           ENTONCES alu.push( MAX_DOUBLE)
                           SINO SI num < 0
                              ENTONCES alu.push(MIN_DOUBLE)
ELEVA          -----    alu.push( Math.pow( alu.pop(), alu.pop() ) )
SEN            -----    alu.push( Math.sin( alu.pop() ) )
COS            -----    alu.push( Math.cos( alu.pop() ) )
TAN            -----    alu.push( Math.tan( alu.pop() ) )
ASEN           -----    alu.push( Math.asin( alu.pop() ) )
ACOS           -----    alu.push( Math.acos( alu.pop() ) )
ATAN           -----    alu.push( Math.atan( alu.pop() ) )
EXP            -----    alu.push( Math.exp( alu.pop() ) )
LOG            -----    alu.push( Math.log10( alu.pop() ) )
LN             -----    alu.push( Math.log( alu.pop() ) )
```

Por ejemplo: si escribimos la expresión -3+4/x nuestra máquina virtual quedará:

datos: [x][-3][4]
programa: [push 1][push 2][push 0][divide][suma]

Al ejecutarlo: (si a x le damos el valor 2: datos[0]= 2;)
sentencia 0: pila = [-3]
sentencia 1: pila = [-3][4]
sentencia 2: pila = [-3][4][2]
sentencia 3: pila = [-3][2] // se ha sacado [4][2] y se mete [4] / [2]
sentencia 4: pila = [-1] // se ha sacado [-3][2] y se mete [-3] + [2]

Esto nos da idea de como ejecutaremos las sentencias, pero para generar el programa que tenemos en el array programa a partir de cualquier expresión, hay que saber un poco como funcionan los lenguajes.

PASO 2 (1 PUNTO). LENGUAJE DE LAS EXPRESIONES Y EL ANALIZADOR LEXICO.

Los lenguajes que se utilizan en informática suelen ser formales (solo interesan ciertas estructuras linguísticas, las que no permiten definir cosas ambigüas). Un lenguaje formal se puede expresar mediante una gramática G.

Una gramática G consiste en 5 cosas G= (NT, T, P, S) que explico aquí:

- NT: conjunto de símbolos no terminales. Son símbolos que permiten definir estados

temporales. No los veremos escritos en los mensajes del lenguaje, pero ayudan a definir la estructura de las expresiones sintácticas que usa. Por ejemplo: PREDICADO ::= VERBO + COMPLEMENTO_VERBAL predicado sería un elemento no terminal de un lenguaje. Suelen escribirse en mayúsculas.

- T (conjunto de símbolos terminales): son los símbolos que si vemos en los mensajes del lenguaje. Por ejemplo un predicado puede ser: volaba muy alto. Se indican en minúsculas.
- P (conjunto de reglas de producción): permite indicar como se puede generar cualquier mensaje del lenguaje. Imagina una gramática con estas reglas:

  T1 → a T2
  T2 → b | b T1 b

En esas regla T1 y T2 son no terminales (en mayúscula) o intermedios. Y a y b son terminales (o finales, y por eso están en minúscula). Estas reglas permiten generar mensajes de tipo:

T1 → a T2 → ab
T1 → a T2 → ab T1 b → abaT2 b → ababb
T1 → a T2 → ab T1 b → abaT2 b → ababT1bb → ...

- **S (es el símbolo origen): el símbolo no terminal a partir del cual se genera cualquier mensaje del lenguaje que define la gramática.**

Si definimos bien una gramática, podemos hacer un programa que vaya recorriendo los mensajes de su lenguaje, identificando sus partes y realizando alguna tarea, como por ejemplo ir generando otras sentencias que sean capaces de calcular el valor de esa expresión. Este programa se denomina analizador sintáctico. Hay muchos tipos, uno de ellos es de los más sencillos y se llaman recursivos.

Para facilitarnos la tarea de reconocer los elementos de un mensaje, iremos identificando los elementos que vayamos encontrando en el mensaje. A estos elementos los llamamos tokens. En nuestro lenguaje un token lo podemos definir así:

```
enum Token { SIGNOMENOS, NUMERO, VARIABLE, SUMA, RESTA, MULTIPLICA, DIVIDE,
             ELEVA, FUNCION, ABREPARENTESIS, CIERRAPARENTESIS, INICIO, FINAL,
             ERROR }
```

El analizador léxico lo podemos definir así:

```
class AnalizadorLexico {
      private String expresion; // La expresión a analizar
      private int longitud;     // cantidad de letras de la expresión
      int posActual;            // Qué letra se va a analizar
      Token tokenAnterior;      // Token que se ha encontrado antes
      String valorToken;        // letras que forman el token actual
      String error;             // Mensaje de error léxico (token mal escrito)

      public AnalizadorLexico(String exp) {
            expresion= exp;
            longitud = (expresion == null)? 0: expresion.length();
```

```
        posActual= 0;
        tokenAnterior = Token.INICIO;
    }

    public Token getToken() {
        char letra=' ';
        // quitar espacios
        while(posActual < longitud && (letra=expresion.charAt(posActual)) == ' ')
            posActual++;
        if (posActual >= longitud) return Token.FINAL;
        // según el valor de letra estaremos en un token u otro
        posActual++;
        switch(letra) {
        case '-': // Si anterior es INICIO O ABREPARENTESIS SIGNOMENOS Sino, RESTA
        case '+': // Si anterior es INICIO O ABREPARENTESIS coge el siguiente
                  // con llamada recursiva porque no hace nada, si no, SUMA
        case 'x': case 'X': // VARIABLE
        case '*':    // MULTIPLICA
        case '/':    // DIVIDE
        case '^':    // ELEVA
        case '0': case '1': case '2': case '3': case '4': case '5': case '6':
        case '7': case '8': case '9': case '.': case ',':
                   boolean separador= (letra == ',' || letra == '.')? true:false;
                     // LOS NUMEROS PUEDEN SER:   1     12 123.   123.0  123,0 .45
                     // LOS NUMEROS NO PUEDEN SER:   1.2.3  2,3,4  2.3,4
        default:  // DEBE SER FUNCION, comprueba que es correcta sino ERROR:
                  // sen, cos, tan, asen, acos, atan, exp, log, ln (mayús. o min)
        }
    }

    public String getValor() { // devuelve cadena del token }

    public String getError() { // devuelve último error léxico  }
}
```

Ejemplos de resultados: si cuando acabéis os hacéis un programa para ver lo que genera podéis probar estos ejemplos:

```
    public static void main(String[] args) {
        System.out.print("Teclee una expresión: ");
        String e = sc.nextLine();
        AnalizadorLexico al= new AnalizadorLexico(e);
        Token t= Token.INICIO;
        while (t != Token.FINAL && t != Token.ERROR) {
            System.out.print( t + " ");
            t= al.getToken();
        }
        System.out.println( t + " ");
        if(t == Token.ERROR) System.out.println("Error Lexico " + al.getError() );
    }
```

```
Teclee una expresión: 3
INICIO NUMERO FINAL
```

```
Teclee una expresión: 3 * x
INICIO NUMERO MULTIPLICA VARIABLE FINAL
```

```
Teclee una expresión: -1 + cos(1/x)
INICIO SIGNOMENOS NUMERO SUMA FUNCION ABREPARENTESIS NUMERO DIVIDE VARIABLE CIERRAPARENTESIS FINAL
```

```
Teclee una expresión: +1 - 4 * x^2
INICIO NUMERO RESTA NUMERO MULTIPLICA VARIABLE ELEVA NUMERO FINAL
```

```
Teclee una expresión: 3+3/2
INICIO NUMERO SUMA NUMERO DIVIDE NUMERO FINAL
```

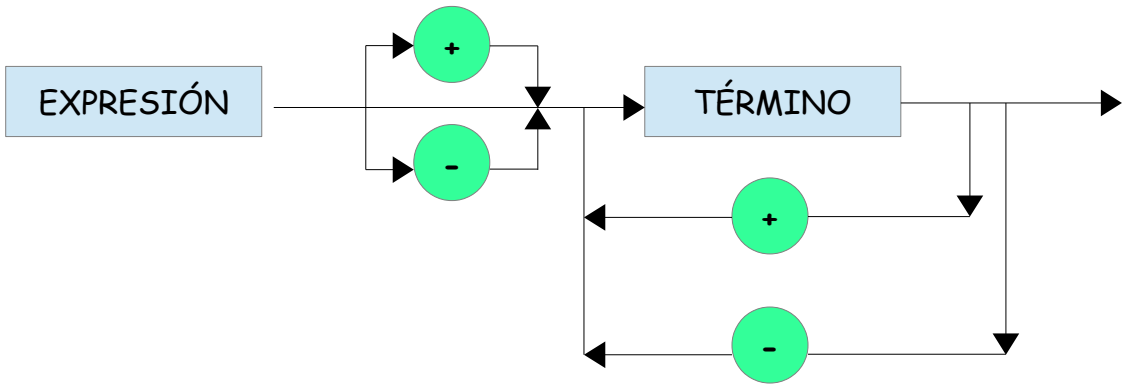
```
Teclee una expresión: (3+3)/2
INICIO ABREPARENTESIS NUMERO SUMA NUMERO CIERRAPARENTESIS DIVIDE NUMERO FINAL
```

PASO 3 (1 PUNTO). EL ÁRBOL SINTÁCTICO DE LAS EXPRESIONES:

Las reglas de producción de una gramática G, define los mensajes bien construidos de un lenguaje. Las reglas se pueden expresar mediante diagramas de alambre (los símbolos no terminales en cuadrados y los terminales en círculos) o como reglas (inicio → combinación de terminales y no terminales). Por ejemplo el lenguaje de las expresiones que usa nuestra calculadora comienzan así porque E (de EXPRESIÓN) es el símbolo inicial de todos los mensajes del lenguaje (la | indica cada alternatica):

E → +T | -T | T | T + T1 | T – T1
T1 → T + T1 | T – T1 | T

En diagrama de alambres:

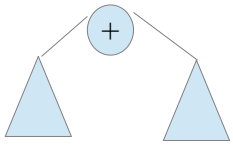


Los objetivos del analizador sintáctico son dos: construir el árbol sintáctico que representa la estructura del mensaje (y que luego podremos procesar para generar las sentencias que sean capaces de hacer los cálculos) y por otro lado, detectar posibles errores sintácticos si el mensaje no sigue ninguna estructura capaz de producir las reglas de producción. Para ello irá pidiendo tokens al analizador léxico y construyendo el árbol hasta que alcance el final del mensaje o encuentre errores. De forma genérica el analizador de esta regla comenzará así (más adelante te daré una versión más real de la misma):

```
public void analiza(){
    Token t= aL.getToken();
    expresion(Token t);   // Comenzamos a analizar por el símbolo inicial
}

public void expresion(Token t) {
   if( t == Token.SIGNOMENOS ) {     // El signo + se ignora porque no cambia nada
       // crear Token.SIGNOMENOS
       t= aL.getToken();
```

```
    }
    termino(t);
    while(t == Token.SUMA || t == Token.RESTA ) {
        // crear Token t
        termino();
    }
}
```

¿Cómo será un árbol sintáctico? En nuestro caso necesitamos que sea un árbol binario donde cada nodo tenga el token que representa y su valor, no todos, porque algunos nos ayudarán a definir la estructura del árbol (los paréntesis) y algunos podremos transformarlos (como el signo menos delante de un número). Los operadores binarios (que trabajan con dos operandos o valores, estarán en la raíz de un árbol y sus valores serán sus hijos (uno izquierda y otro derecha).



Nuestra clase será este código donde he dejado todas las variables miembro públicos por eficiencia:

```
class ArbolSintactico {
        public Token t;
        public String valor;
        public ArbolSintactico hd, hi;

    public ArbolSintactico(Token token, String s, ArbolSintactico i, ArbolSintactico d) {
        t= token;
        s= valor;
        hi= i;
        hd= d;
    }

}
```

La clase AnalizadorSintactico y un programa más realista, debe ocuparse además de seguir la estructura de las reglas de producción de la gramática, en ir construyendo el árbol sintáctico:

```
....
static public void main()
        Scanner s = new Scanner( System.in );
        Pila programa = new Pila( enum TOKEN);
        System.out.print("Teclee una expresión: ");
        String e = sc.nextLine();
        AnalizadorSintactico as = new AnalizadorSintactico(e);
        ArbolSintactico= as.analiza();
}

class AnalizadorSintactico {
String expre;
AnalizadorLexico aL;
ArbolSintactico arbol;
String ultimoError;

public AnalizadorSintactico(String expresion) {
        expre= expresion;
        aL = new AnalizadorLexico(expre);
}
```

```
public ArbolSintactico analiza(){
    Token t;
    arbol = new ArbolSintactico();
    t= aL.getToken();
    arbol= expresion(Token t); // Comenzamos a analizar por el símbolo inicial
}

public ArbolSintactico expresion(Token t) {
   ArbolSintactico raiz, nuevo;
   if( t == Token.SIGNOMENOS ) {               // El signo + se ignora
      raiz = new ArbolSintactico(Token.SIGNOMENOS,null,null,null );
      t= aL.getToken();
   }
    if (raiz != null) raiz.hi = termino(t); else raiz = termino(t);
   while(t == Token.SUMA || t == Token.RESTA ) {
        nuevo= new ArbolSintactico(t, 0, raiz, null);
        raiz= nuevo;
       raiz.hd= termino();
   }
   return raiz;
}
```

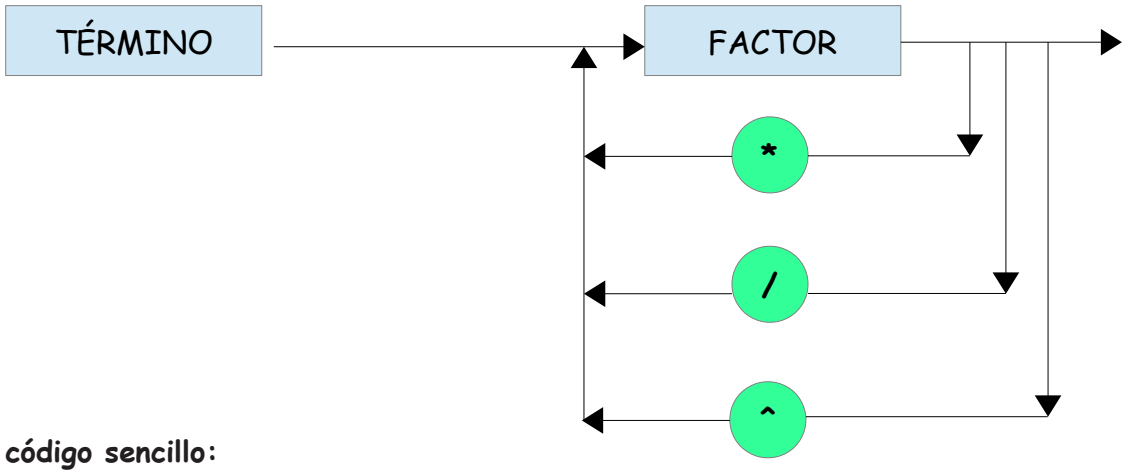
Para TERMINO (T), la regla genera elementos factores. Si vemos la regla en forma de expresiones, en diagrama de alambres y en programa sin entrar en detalles:

Reglas:

T → F | F * F1 | F / F1 | F ^ F1

F1 → T

Diagrama de alambres:



El código sencillo:

```
public void termino(Token t) {
   factor(t);
   while(t == Token.MULTIPLICA || t == Token.DIVIDE || t == Token.ELEVA ) {
       // crear Token t
       factor();
   }
}
```

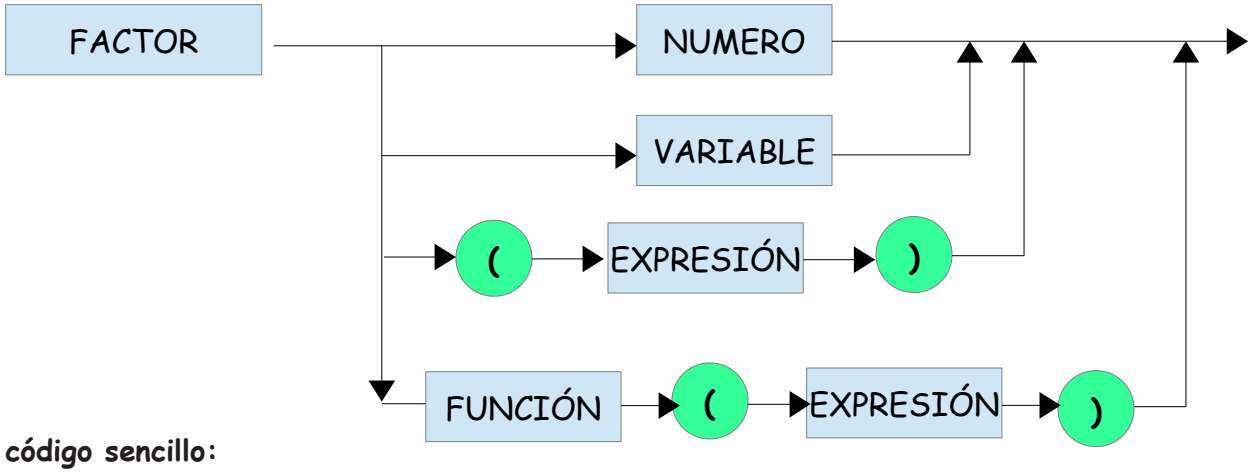
Y FACTOR (F) se expande a elementos más cercanos a símbolos terminales. Ahora

además de construir el árbol habrá que recordar los valores de los tokens:

Reglas:

F → N | V | (E) | F(E) // N es número, V es variable, F función, E expresión

Diagrama de alambres:

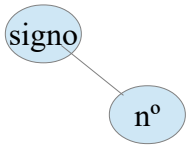


El código sencillo:

```
factor(Token t) {
   if(t == NUMERO)  return árbol número;
   if(t == VARIABLE) return árbol variable;
   if(t == ABREPARENTESIS) {
       t= getToken();
       expresion(t);
       t = getToken();
       if( t != CIERRAPARENTESIS) devuelve error
   }
   if(t == FUNCION)  {
       t= aL.getToken();
       if(t != ABREPARENTESIS) devuelve error;
       t= getToken();
       e= expresion(t);
       t = getToken();
       if( t != CIERRAPARENTESIS) devuelve error
       devuelve e;
   }
   devuelve error
}
```

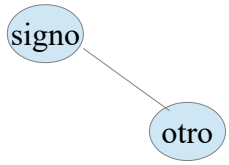
PASO 4 (1 PUNTO). GENERAR EL PROGRAMA A PARTIR DEL ÁRBOL SINTÁCTICO.

Una vez generado el árbol, queda traducir su estructura a una serie de sentencias que podamos ejecutar:

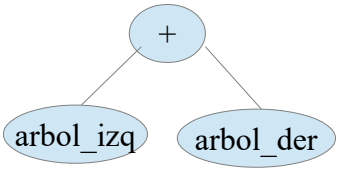


es como tener -nº

datos[dato++] = -nº
programa[sentencia++] = (push, dato-1)



programa[sentencia++] = otro...
programa[sentencia++] = (cambiasigno,0)



programa[sentencia++] = arbol_izq
programa[sentencia++] = arbol_der
programa[sentencia++]= (suma, 0)

Es decir, salvo si el nodo es un cambio de signo (que la expresión estará siempre a la derecha, o la izquierda, lo que tu decidas) el resto de sentencias se recorren en inorden: primero el subárbol de la derecha, luego el subárbol de la derecha y por último se genera el nodo de la sentencia.
Tras ejecutar el método generaPrograma(ArbolSintactico, programa, datos, cpu) todo quedará listo para ejecutar cualquier expresión válida si más que ir recorriendo el programa desde la primera sentencia hasta la última y si hay variables en la expresión, sustituir el valor que van a tomar en el array de datos. Observa que datos y sentencias nunca cambian en cada ejecución salvo el valor de x (en la posición 0 de los datos).

```
Enum Sentencia { PUSH, SUM, REST, SIGN, MULT, DIV, ELEV, SEN, COS, TAN,
ASEN, ACOS, ATAN, EXP, LOG, LN }

class Operacion{  Sentencia s;   int dato; }
List<double> datos;
List<Operacion> programa;
Pila<double> alu;

        PUSH            pos         alu.push( datos[pos] )
        SUMA            -----       alu.push( alu.pop()  + alu.pop() )
        CAMBIASIGNO     -----       alu.push( - alu.pop() )
        RESTA           -----       alu.push( alu.pop()  - alu.pop() )
        MULTIPLICA      -----       alu.push( alu.pop()  * alu.pop() )
        DIVIDE          -----       num ← alu.pop()     den ← alu.pop()
                                    SI den != 0
                                    ENTONCES  alu.push( alu.pop()  / alu.pop() )
                                    SINO
                                        Warning →
                                        SI num > 0
                                        ENTONCES alu.push( MAX_DOUBLE)
                                        SINO SI num < 0
                                            ENTONCES alu.push(MIN_DOUBLE)
        ELEVA           -----       alu.push( Math.pow( alu.pop(), alu.pop() ) )
```

SEN	-----	alu.push(Math.sin(alu.pop()))
COS	-----	alu.push(Math.cos(alu.pop()))
TAN	-----	alu.push(Math.tan(alu.pop()))
ASEN	-----	alu.push(Math.asin(alu.pop()))
ACOS	-----	alu.push(Math.acos(alu.pop()))
ATAN	-----	alu.push(Math.atan(alu.pop()))
EXP	-----	alu.push(Math.exp(alu.pop()))
LOG	-----	alu.push(Math.log10(alu.pop()))
LN	-----	alu.push(Math.log(alu.pop()))

LA CALCULADORA (1 PUNTO): ahora hay que unir todo esto y darle una interfaz. Crea cuadros de texto para escribir expresiones, límite inferior donde comenzar a dibujar, límite superior y un botón para iniciar el dibujo.